Test and Evaluation

Automated integration tests ran against actual MySQL 8.4.7 and an HTTP Node.js server. These results evaluate technical behavior; they are not participant feedback or a substitute for browser usability testing. The same sixteen tests also passed after creating a fresh database. Detailed executable steps and assertions are in tests/system.test.js.

Test ID	Requirement or scenario	Actual result	Evidence
T01	Registration and login	PASS	production-tests.txt
T02	Valid empty and invalid search	PASS	production-tests.txt
T03	Authentication and CSRF rejection	PASS	production-tests.txt
T04	Server fare and complete persistence	PASS	production-tests.txt
T05	Customer ownership	PASS	production-tests.txt
T06	Concurrent same-seat requests	PASS	production-tests.txt
T07	Multi-passenger rollback	PASS	production-tests.txt
T08	Cancellation and inventory release	PASS	production-tests.txt
T09	Malformed passenger and seat	PASS	production-tests.txt
T10	Administrator authorization	PASS	production-tests.txt
T11	Immutable booked fare	PASS	production-tests.txt
T12	Analytics reconciliation	PASS	production-tests.txt
T13	Airport and aircraft maintenance	PASS	production-tests.txt
T14	Full capacity and schedule cancellation	PASS	production-tests.txt
T15	Logout revocation	PASS	production-tests.txt
T16	MySQL FK and check constraints	PASS	production-tests.txt

T17 separately checks server-restart persistence; its saved output is evidence/persistence-check.txt. If that file is absent or failing, the criterion is pending. Tests use fictional accounts and remove their own test bookings afterward.

Critical Integrity Test Procedures

Test	Steps and expected result	Observed result
T06 concurrency	Two authenticated users submit the same free seat concurrently. Expect one HTTP 201, one HTTP 409 and one active allocation.	Passed: one accepted request and one conflict; count equals one.
T07 rollback	Submit two passengers: first requests a free seat; second requests an occupied seat. Expect failure with unchanged booking and passenger counts.	Passed: no partial booking or passenger records.
T08 cancellation	Cancel an owned confirmed booking. Retrieve status, inspect payment, and query seat availability.	Passed: cancelled, voided and seat available.
T11 fare change	Save booking totals, edit departure fare, read totals again, restore fare.	Passed: existing totals unchanged.
T14 full flight	Create a future departure; fill all 24 seats in groups of four; try another allocation; cancel departure.	Passed: extra booking rejected; all active seats released and bookings cancelled.
T17 restart	Create booking and session, stop Node.js, start it again, retrieve and cancel with saved session.	See separate persisted execution output.

Defects and retests

A production test run failed because a database process in a different sandbox held the data directory and was unreachable. The preview context was stopped and the same suite was rerun in a shared execution context; all sixteen tests passed. This was not counted as a successful run. The initial test for rejected database writes was reviewed to ensure database connectivity had already been proven by the preceding tests.

Manual checks still required

Verify desktop and narrow mobile layouts, keyboard-only navigation, error announcements, print output, and the full customer/admin browser journeys. Capture real screenshots. Browser access from this session timed out, so these checks are pending rather than assumed to pass.

Three User Evaluation Plan

Recruit three different real participants and identify them as P01, P02 and P03. Explain that all data is fictional. Give each participant the same tasks without coaching unless they are blocked. Do not record real names or sensitive account information. At least one participant must review administration using the demo administrator account.

Task	Record
Search a flight	Route/date chosen; completion; confusion; elapsed time if measured
Create a reservation	Passenger entry; seat selection; errors; assistance needed
Retrieve confirmation	Reference retrieval; clarity of fare/status; completion
Cancel reservation	Completion; understanding of released seats; error messages
Administrator review by at least one user	Create or edit a schedule; inspect reservation; role clarity

Ratings and feedback

After the tasks, collect ratings from 1 poor to 5 excellent for ease of use, clarity, reliability and overall satisfaction. Record the participant's actual comments verbatim where possible. Ask which one change would help most. Capture version, date, device/browser and any assistance. Use the separate User_Evaluation_Forms.pdf.

Participant	Session status	Actual ratings and comments
P01	Not conducted	Not collected
P02	Not conducted	Not collected
P03	Not conducted	Not collected

No participant, rating, completion time or comment has been invented. The final evaluation must be updated after real sessions.

Evaluation Against Success Criteria

Criterion	Current evidence	Status
Correct fare calculation	T04 and T11	Verified by automated tests
Persistent bookings	MySQL integration plus restart check output	Review T17 evidence
Duplicate-seat prevention	T06 plus database unique constraint	Verified
Atomic multi-passenger booking	T07 unchanged row counts after conflict	Verified
Cancellation consistency	T08 and T14	Verified
Role and ownership separation	T03, T05, T10 and T15	Verified
Fresh environment setup	clean-setup.txt	Verified on new database
Usable customer/admin browser flows	Browser checks and screenshots	Pending
Three real users complete tasks	Participant forms and actual feedback	Pending

How to close the evaluation

For each real issue, add a defect ID, reproduction steps, expected and actual behavior, severity, correction and retest evidence. Update the implementation change log and affected requirements when a fix changes behavior. Do not replace earlier evidence silently. If the release changes, create a new commit and record the tested commit hash. Summarize remaining limitations and compare participant task completion with the original success criteria before submission.